



UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN
FACULTAD DE CIENCIAS FÍSICO
MATEMÁTICAS

Análisis Numérico para Programación

Proyecto 1:

Solución de sistemas de ecuaciones por método
Gauss Seidel

Alumna: Kelly Yiseth Cano Montiel

Matricula: 1979822

Grupo: 033

Introducción

El método de Gauss-Seidel es un procedimiento iterativo empleado para la resolución de sistemas de ecuaciones lineales. Su funcionamiento se basa en tratar de aproximarse progresivamente a la solución del sistema mediante sustituciones sucesivas, utilizando en cada paso los valores más recientes obtenidos para las incógnitas. De esta manera, se logra una convergencia más rápida en comparación con otros métodos iterativos como el de Jacobi, siempre y cuando se cumplan condiciones como la diagonal dominante de la matriz del sistema.

El algoritmo inicia con una estimación inicial de las variables generalmente usando cero para todas las incógnitas, y en cada iteración, recalcula los valores uno por uno, actualizando inmediatamente las aproximaciones ya obtenidas. Esto se repite hasta que el error entre iteraciones consecutivas sea menor a la tolerancia definida o hasta alcanzar un número máximo de iteraciones.

Para programar este método se usó la siguiente fórmula que muestra el procedimiento de la iteración $k+1$ sin necesidad de despejar para cada incógnita del sistema.

$$x_i^{(k+1)} = \frac{1}{a_{i,i}} (b_i - \sum_{j=1}^{i-1} a_{i,j} x_j^{(k+1)} - \sum_{j=i+1}^n a_{i,j} x_j^{(k)}); \quad i = 1, 2, \dots, n; \quad k = 0, 1, 2, \dots$$

Esta fórmula se obtiene directamente de la fórmula de Jacobi separando la sumatoria en dos partes: los componentes que aún no han sido calculados se los toma de la iteración anterior k , mientras que los ya calculados, se los toma de la iteración $k+1$ para después restar las sumatorias y b_i que representa el término independiente perteneciente a x_i que sería la incógnita que se está intentado despejar, y finalmente se divide entre el elemento de la diagonal principal $a_{i,i}$.

Codificación

```
"""Resolución de un sistema de ecuaciones por el metodo gauss seidel"""
import numpy as np

n = int(input("ingrese el numero de ecuaciones: "))
B = np.zeros((n,1), dtype=float)
matriz = np.zeros((n,n), dtype=float)
x_k1 = np.zeros((n,1),dtype=float)

for i in range(0,n):
    while True:
        ecuacion = input(f"Ingrese los coeficientes de la ecuacion {i+1} separados por espacios: ").split()
        if len(ecuacion) != n:
            print(f"¡Error! se deben ingresar {n} coeficientes")
            continue
        try:
            ecuacion_float = []
            for valor in ecuacion:
                ecuacion_float.append(float(valor))
            matriz[i] = ecuacion_float
            break
        except ValueError:
            print("¡Error! solo se deben ingresar numeros")
    while True:
        b = input(f"Ingrese el termino independiente de la ecuacion {i+1}: ").split()
        if len(b) != 1:
            print("Error solo se debe ingresar un numero")
            continue
        try:
```

Primero se pide el tamaño de la matriz $n \times n$ al usuario, y de acuerdo al tamaño ingresado se crean las matrices de coeficientes, términos independientes e incógnitas para después preguntar por cada ecuación del sistema y validarlas que tengan exactamente n términos.

```

        B[i][0] = float(b[0])
        break
    except ValueError:
        print("Solo debe ingresar numeros")

print("\nDatos ingresados:")
for filaA, filaB in zip(matriz, B):
    print(filaA, " ", filaB)
print("-----")
#verificando si la matriz es diagonalmente dominante
flag = False
for i in range(n):
    suma_diagonal = 0
    for j in range(n):
        if i != j:
            suma_diagonal = suma_diagonal + abs(matriz[i][j])
    if suma_diagonal > abs(matriz[i][i]):
        print("***La matriz no es diagonalmente dominante**")
        flag = True
        break

```

```

if flag:
    print("Iniciando pivoteo parcial por filas...")
    for i in range(n):
        columna = abs(matriz[i:, i])
        indice = np.argmax(columna)
        max_indice = indice + i
        if max_indice != i:
            matriz[[i, max_indice]] = matriz[[max_indice, i]]
            B[[i, max_indice]] = B[[max_indice, i]]
            print(f"Se intercambiaron filas {i} y {max_indice}")
    verificar_dominante = False
    for i in range(n):
        suma_diagonal = 0
        for j in range(n):
            if i != j:
                suma_diagonal = suma_diagonal + abs(matriz[i][j])
        if suma_diagonal > abs(matriz[i][i]):
            verificar_dominante = True
            break
    if verificar_dominante:
        print("\nDespués del pivoteo parcial, la matriz aun no es diagonalmente dominante.")
        print("El método de Gauss-Seidel podría no converger.")
        respuesta = input("¿Desea continuar? ('si'/'no')")
        if respuesta == 'no':
            exit()

```

Después se verifica que la matriz ingresada sea diagonalmente dominante, esto se hace seleccionando la primera columna y recorriéndola de arriba abajo por filas para saber cuál es de mayor valor absoluto, y así se sigue con las demás columnas, para después verificar si ya es diagonalmente dominante.

```

k = int(input("Ingrese el maximo de iteraciones: "))
e = float(input("Ingrese el error deseado: "))
error_aproximado = np.zeros((n,1),dtype=float)
tolerancia = False
iteracion = 0
while iteracion < k and not tolerancia:
    x = x_k1.copy()
    for i in range(n):
        sumatoria1 = 0
        sumatoria2 = 0
        for j in range(i):
            sumatoria1 = sumatoria1 + matriz[i][j] * x_k1[j]
        for j in range(i+1,n):
            sumatoria2 = sumatoria2 + matriz[i][j] * x[j]
        x_k1[i] = (B[i] - sumatoria1 - sumatoria2) / matriz[i][i]

        error_aproximado[i][0] = np.abs((x_k1[i][0] - x[i][0]) / (x_k1[i][0] + 1e-10))
    tolerancia = np.max(error_aproximado) <= e
    iteracion += 1

if tolerancia:
    print("\n**Vector solución: \n",x_k1,"\n\n**Encontrado en la iteración: ",iteracion)
else:
    print(f"No se alcanzó la tolerancia en {k} iteraciones. Última aproximación: \n {x_k1}")

```

Para empezar con el método Gauss seidel se pide el error esperado y las iteraciones máximas para llegar a este margen de error. El ciclo while se repite solo si la iteración a seguir es menor al límite de iteraciones, y además la tolerancia sea mayor al error esperado ingresado.

Se crea una matriz X donde se copiarán los valores de K antes de que se actualicen a K+1 y posteriormente se inician dos ciclos for, uno para recorrer la matriz. La sumatoria 1 calcula de acuerdo a los valores de K+1 empezando con i = 0 y con j desde cero hasta el valor actual de i, la sumatoria 2 suma tomando los valores de K que no han sido calculados en la iteración actual, por eso toma el ciclo for desde i+1 hasta n. Finalmente se calcula el error aproximado para verificar si ya se llegó al valor deseado.

Ejecución

```
PS C:\Users\Dell> & C:/Users/Dell/AppData/Local/Programs/Python/Python313/
ingrese el numero de ecuaciones: 3
Ingrese los coeficientes de la ecuacion 1 separados por espacios: 4 1 2
Ingrese el termino independiente de la ecuacion 1: 16
Ingrese los coeficientes de la ecuacion 2 separados por espacios: 1 3 1
Ingrese el termino independiente de la ecuacion 2: 10
Ingrese los coeficientes de la ecuacion 3 separados por espacios: 1 2 5
Ingrese el termino independiente de la ecuacion 3: 12

Datos ingresados:
[4. 1. 2.]      [16.]
[1. 3. 1.]      [10.]
[1. 2. 5.]      [12.]
-----
Ingrese el maximo de iteraciones: 5
Ingrese el error deseado: 0.05

**Vector solución:
[[3.00833333]
 [2.00833333]
 [0.995      ]]

**Encontrado en la iteración: 3
PS C:\Users\Dell> 
```

Conclusión

El desarrollar el método de Gauss-Seidel en lenguaje Python permitió comprender de manera práctica y clara cómo funciona la implementación de este método para la resolución de sistemas de ecuaciones lineales gracias a su algoritmo. Durante el desarrollo resultó evidente la importancia de verificar la condición de la diagonal dominante es decir que cada elemento perteneciente a la diagonal principal debe ser mayor o igual al valor absoluto de la suma de sus elementos restantes en dicha fila. Para lograr que la matriz sea diagonalmente dominante se usó la técnica de pivoteo parcial, y así buscar garantizar la estabilidad y la convergencia del método.

Además, se usaron las estructuras de control, operaciones matriciales y la librería NumPy lo cual facilitó en gran manera la programación y optimización de los cálculos. En conclusión, este trabajo no solo reafirmó los fundamentos teóricos del método, sino que también ayudó a desarrollar nuevas habilidades y conocimiento sobre Python y las herramientas que ofrece para trabajar de manera versátil y eficiente en la resolución de problemas matemáticos.

Bibliografía

- Rodríguez Ojeda, Luis. Análisis numérico básico: un enfoque algorítmico con el soporte de Python. Versión 4.3